Islamabad AQI Prediction System

A Project Story from Confusion to Completion

Fatiha Maryam

10Pearls Shine Internship | June 2026

https://github.com/Fatiha-maryam/Islamabad_aqi_prediction

Project	Islamabad AQI 72-Hour Prediction System
Tech Stack	Python, MongoDB Atlas, MLflow, DagsHub, GitHub Actions, Streamlit
Models	XGBoost, LightGBM, CatBoost, RandomForest, StackingRegressor
Best Result	24h R^2 = 0.817 | 48h R^2 = 0.674 | 72h R^2 = 0.574
Live Dashboard	**https://islamabad-aqi-forecast.streamlit.app**
Timeline	April 15 - May 31, 2026

1. Where It All Began - The Blank Page Problem

I started this internship project on April 15, 2026, and for almost the first month, I had no idea what I was actually doing. The task sounded straightforward on paper, predict Islamabad's Air Quality Index for the next three days. But when I sat down and opened my laptop, I was staring at a blank screen with a hundred questions and no idea where to start.

Should I do EDA first? Should I build pipelines? What even is a feature pipeline? How do features get stored? Will I need to manually add rows every single time? How does everything connect together? These were not deep technical questions, they were, as I later realized, very basic ones. But at that point, they felt enormous, and every answer seemed to lead to five more questions.

The most embarrassing confusion I had early on, and I admit this freely, was about prediction itself. I kept thinking: ***if I want to predict tomorrow's AQI, I need tomorrow's pollutant values.***

But how can I know tomorrow's pollutants? *It sounds almost funny now, but at the time it felt like a real wall. I genuinely did not understand how time series forecasting worked, that you use the past to predict the future, not the other way around.*

That was the starting point. Complete confusion, a lot of fear, and a blank screen. What followed was one of the most frustrating, rewarding, and educational experiences of my life.

2. The PDF Nightmare - Three Weeks I Will Never Forget

Before even thinking about models, I needed data. And I wanted it to be reliable. I did not trust random APIs, I wanted official, sensor-verified data from Pakistan's own environmental agency.

I found that the Pakistan Environmental Protection Agency publishes monthly AQI and pollutant readings collected from real monitoring stations across the country. They publish it as PDFs. Perfect, I thought. Official government data, real sensors, nothing better than this.

So I started scraping. I collected PDFs going back to 2023, all the way to March 2026. I wrote scripts to extract the tables, clean the text, parse the numbers. It took days to get the extraction working properly. I was genuinely proud when I finally had a large CSV file sitting in front of me.

Then I looked at the data.

Missing values everywhere. Entire columns with nulls. Stations that simply stopped reporting for weeks at a time. Some months had data for three pollutants, some had seven. The schema was inconsistent across months. Rows that should have been daily had gaps of four or five days with no explanation.

I spent **nearly three full weeks** trying to clean this data. I tried interpolation, forward fill, median imputation, and combinations of all three. The problem was not just the missing values, it was that the missingness itself was not random. Stations went offline during pollution spikes, which is exactly when you need the data most. Any imputation I applied was introducing bias into the most important parts of the dataset.

I tested model after model on this data. The results were consistently bad. R^2 values hovering around 0.2 to 0.3. I thought maybe my feature engineering was wrong. I tried different lag configurations. I tried different models. The data itself was the problem, but I could not see that yet.

Eventually, after weeks of frustration, I made the decision to abandon the PDF dataset entirely and switch to API-based data fetching. It felt like giving up three weeks of work. But it was the right call, and honestly one of the most important decisions of the entire project. Sometimes knowing when to stop and restart is the real skill.

3. Starting Fresh - The Open-Meteo API Discovery

After abandoning the PDF approach, I researched alternative data sources and found Open-Meteo, a free, open-source weather and air quality API that provides hourly historical data for any coordinates in the world. No API key required. No payment. Just clean, reliable, hourly data.

I tested it immediately. Fetched one month of data for Islamabad coordinates (33.7294°N, 73.0931°E). The data came back complete, consistent, and hourly. PM2.5, PM10, NO2, Ozone, CO, US AQI, all there, no gaps.

I made two API calls: one to **air-quality-api.open-meteo.com** for pollutant data and one to **archive-api.open-meteo.com** for weather data (temperature, humidity, wind speed, precipitation). These two sources combined gave me everything I needed.

I initially fetched daily data, got results, performed manual EDA on Kaggle, then realized hourly data would give me much more to work with, both in terms of volume and the ability to create lag features at a fine-grained level. So I switched to hourly, re-fetched everything, and started building properly.

3.1. The Long Middle - Daily Data, Bad Results, and Research Rabbit Holes

Switching to Open-Meteo was not an instant solution. What followed was another long stretch of trial, error, and genuine confusion.

My first attempt was with OpenWeatherMap. I collected data from 2025 to 2026 and ran EDA on it. The results were disappointing. I did not understand why at the time, but the data coverage was limited and the features I was using were too simple to capture any meaningful patterns.

So I switched to Open-Meteo and fetched daily data from 2023 to 2026. Better source, longer history. I spent significant time on EDA again, built models, and the results were still not good. The R^2 values were low across all horizons. I knew something was wrong with my approach but could not pinpoint what.

The feature engineering step was where I got truly stuck. I knew I needed lag features but did not understand how to design them properly for my specific problem. I spent weeks reading research papers on air quality forecasting, watching YouTube videos on time series feature engineering, studying other people's Kaggle notebooks. Every time I felt I understood the concept, I would open my own project and go completely blank again. The gap between understanding something in theory and applying it to your own data is real and humbling.

Honestly, I did not even know that hourly AQI data was available or that it would make such a difference. Moving from daily to hourly was not a planned decision, it came from experimentation and gradually realizing that more granular data gave the model much more to learn from.

By the end of May, after all this back and forth, I made a decisive move. I deleted everything stored in MongoDB, cleared the slate entirely, and started the backfill fresh, this time with 2024 to 2026 hourly data, redesigned features, and a much clearer understanding of what I was actually trying to do. That reset is what ultimately led to the results I am proud of.

4. The EDA Phase - Learning What the Data Was Trying to Say

With clean hourly data in hand, I ran extensive exploratory data analysis on Kaggle. This is where things started to click.

The seasonal patterns were immediately visible. January and February showed AQI values consistently above 150, deep into the unhealthy range. This matched what I knew about Islamabad's winter smog, caused by temperature inversions that trap pollutants close to the ground. Summer months showed lower AQI, partly because monsoon rains wash particulate matter out of the air.

The hourly patterns were also clear. Morning rush hours (7-9 AM) and evening hours (5-7 PM) consistently showed AQI spikes. Weekdays were worse than weekends in the commercial areas.

The most important discovery from EDA was the relationship between **PM2.5 and US AQI.** PM2.5 fine particulate matter smaller than 2.5 micrometers ,is the dominant driver of the US AQI index in Islamabad. When PM2.5 goes up, AQI goes up, almost linearly. This told me that any model serious about predicting AQI must treat PM2.5 as a first-class feature.

I also struggled a lot with understanding lag features during this phase. The concept felt abstract why would I care what AQI was 72 hours ago? But once I visualized the autocorrelation plots, it became obvious. AQI is strongly correlated with its own past values. Today's air quality is heavily influenced by yesterday's air quality. A lag feature is just a way of giving the model access to that historical context.

5. Feature Engineering - Turning Raw Data Into Model Inputs

Feature engineering was where I invested the most thought. I designed four categories of features:

Lag Features

AQI values from 1, 2, 3, 6, 12, 24, 48, and 72 hours ago. These give the model a memory of recent conditions. The 24-hour lag is particularly important because AQI tends to follow daily cycles.

Rolling Statistics

6-hour, 12-hour, and 24-hour moving averages of AQI, plus 12-hour rolling standard deviation. Moving averages smooth out noise and reveal trends. Standard deviation captures volatility, is the AQI stable or jumping around?

Trend Features

The difference between current AQI and AQI 3, 6, and 24 hours ago. This tells the model whether conditions are improving or worsening, which is crucial for forecasting direction.

Seasonal and Time Features

Hour of day encoded as **sine and cosine** to capture the cyclical nature of daily patterns without the artificial jump between hour 23 and hour 0. Season encoding (Winter, Spring, Summer, Autumn), rush hour indicator, and winter smog season flag.

Pollutant Lag Features

PM2.5 from 24 hours ago and PM2.5 12-hour rolling average, because PM2.5 is the strongest individual predictor of AQI.

6. Thought It Was Done - Then Discovered the MongoDB Bug

This is the part of the project I am most proud of, not because it went well, but because of how I handled it going wrong.

By mid-May, I had everything working. Feature pipeline running on GitHub Actions every 5 hours. Training pipeline running daily. Models saved. Dashboard deployed locally. I thought the project was essentially complete. I even showed it to my teacher and received positive feedback.

Then I checked MongoDB. The document count had not changed in days. Still **3,120 rows**, the original historical backfill. Not a single new row had been added by the incremental pipeline despite it running every 5 hours and showing green checkmarks.

I dug into the logs and found the problem. The incremental mode was fetching 80 hours of data, but then creating lag features (which require 72 previous rows) and target variables (which require

72 future rows). 80 minus 144 equals negative 64. Zero rows survived the dropna step. The pipeline was running successfully and storing nothing.

The fix required two things. First, expand the fetch window to 4 days so there is always enough historical context. Second, change the dropna logic to only drop rows missing feature values, not target values, because for live data, targets are unknowable until time passes. A separate update function would retroactively fill in targets once the future data arrived.

This was a significant architectural change. It required rewriting part of the feature pipeline, adding a target update function, and updating the training pipeline to filter out rows with null targets before training. It took two full days to implement and test correctly.

7. The Decision to Improve - Adding More Data and Better Features

After fixing the MongoDB bug, I faced a choice. The project was working. The models had reasonable scores 24-hour R^2 around 0.62. I could submit as-is.

But I knew the results could be better. The original dataset was only 5 months of data 3,120 rows. For a time series model predicting complex atmospheric phenomena, that is quite limited. I decided to invest more time.

I extended the historical backfill to include **January 2024 through May 2026**, over two full years of hourly data. This brought the total to **21,072 rows,** nearly seven times the original dataset. I also designed better features based on what SHAP analysis had revealed: trend features, seasonal encoding, and PM2.5 lag features.

The results were dramatic. 24-hour R^2 jumped from 0.62 to 0.817. 48-hour R^2 went from 0.31 to 0.674. 72-hour R^2 went from 0.18 to 0.574. CatBoost dominated the 24h and 48h predictions. StackingRegressor won the 72h horizon by combining RMSE and R^2 advantages.

This taught me something important about machine learning: more data often beats more clever algorithms. The same models, trained on 7x more data with better features, produced results in a completely different league.

8. The Technical Architecture - How Everything Connects

Looking back, the system I built has clean, professional architecture that I am genuinely proud of:

Data Layer - MongoDB Atlas

MongoDB Atlas (free M0 cluster, Mumbai region) serves as the feature store. Every row contains the engineered features for one hour of data, plus target variables filled in retroactively as future data arrives. Total: 21,000+ documents growing automatically.

Ingestion Layer - Feature Pipeline

Runs every 5 hours via GitHub Actions. Fetches fresh data from two Open-Meteo API endpoints, engineers 35 features, stores new rows in MongoDB (skipping duplicates via unique index on datetime), and updates target values for older rows where future data has now become available.

Training Layer - Training Pipeline

Runs once daily at 2 AM UTC via GitHub Actions. Loads all complete rows from MongoDB, performs chronological 80/20 train-test split (no shuffle - time series integrity), trains 5 models per horizon, selects best via majority vote across MAE, RMSE, and R^2, and registers the winning models in DagsHub MLflow Model Registry.

Registry - DagsHub + MLflow

Three registered models: aqi_model_24h, aqi_model_48h, aqi_model_72h. Each run creates a new version, so the full training history is preserved. All experiment metrics and parameters are logged and visible in the DagsHub UI.

Presentation Layer - Streamlit Dashboard

Publicly deployed at **https://islamabad-aqi-forecast.streamlit.app**. Loads models directly from DagsHub registry, fetches latest features from MongoDB, generates predictions, and displays current AQI, 3-day forecast with dates, health alerts, AQI trend chart, model comparison metrics, and feature importance analysis.

9. Results - Final Model Performance

Horizon	Best Model	MAE	RMSE	R^2
24 Hours	CatBoost	10.65	14.30	**0.817**
48 Hours	CatBoost	14.20	19.04	**0.674**
72 Hours	StackingRegressor	16.81	21.73	**0.574**

10. What I Learned - The Real Takeaways

Data Quality Beats Everything

Three weeks spent on bad data taught me more than any textbook could. No algorithm, no matter how sophisticated, can fix fundamentally biased or missing data. The first and most important question in any ML project is: where does this data come from, and can I trust it?

More Data Often Beats Better Algorithms

Going from 3,120 rows to 21,072 rows improved my R^2 more than weeks of feature engineering and hyperparameter tuning combined. This is a lesson I will carry into every future project.

Build Systems, Not Just Models

A model sitting in a Jupyter notebook is not a project. A model that retrains itself every day, updates its data every 5 hours, registers new versions automatically, and serves predictions through a live public dashboard - that is a system. Building the system was harder than building the model, and also more valuable.

Green Checkmarks Can Lie

My feature pipeline ran successfully for weeks and showed green checkmarks every time. It was also storing zero new rows. Success indicators at the workflow level tell you that the code ran, they do not tell you that the code did what you intended. Always check the actual output.

Fear Is Normal

I was scared at every major decision switching from PDF to API, adding DagsHub, extending the backfill, changing the feature engineering. Every one of these changes risked breaking something that was already working. But every one of them also moved the project forward significantly. Fear is not a signal to stop. It is a signal that you are about to do something that matters.

11. Closing - From a Blank Screen to a Live System

On April 15, 2026, I opened my laptop with no clear idea of what I was supposed to build or how. On May 31, 2026, I have a fully automated, production-ready AQI prediction system for Islamabad that:

Fetches live data from external APIs every 5 hours without any manual intervention. Stores engineered features in a cloud database that grows automatically. Retrains machine learning models every day with the latest available data. Registers the best models in a professional model registry with full version history. Serves predictions through a live public dashboard accessible to anyone in the world.

The final 24-hour model achieves **$R^2 = 0.817$** - meaning it explains over 81% of the variance in Islamabad's air quality one day ahead. That number, to me, is not just a metric. It is proof that the confusion, the frustration, the three wasted weeks, the MongoDB bug, and every small problem along the way were all worth it.

Fatiha Maryam | 10Pearls Shine Internship | May 2026